Python & Algorithms for Coding Interviews

Section 4 — Pattern Recognition System

Reading a problem and immediately knowing which pattern to apply is the single most valuable skill in coding interviews. This section gives you a systematic, repeatable process to do exactly that — every time.

4.1 Problem → Pattern Keyword Map

These are the exact words and phrases that appear in LeetCode problems. When you spot them, your brain should immediately fire the associated pattern.

Array and String Keywords

Keyword / Phrase in Problem	→ Pattern to Use
sorted array, find pair/triplet, sum to target	Two Pointers
subarray, substring, contiguous, window of size k	Sliding Window
longest/shortest subarray satisfying condition	Sliding Window (variable)
range sum, sum of subarray, running total	Prefix Sum
subarray sum equals k, count subarrays with sum	Prefix Sum + Hash Map
frequency, count, anagram, duplicate, most common	Hash Map / Counter
two sum, find complement, find pair	Hash Map
sorted array, find target, search	Binary Search
minimum/maximum satisfying a condition (answer range)	Binary Search on Answer Space
next greater, next smaller, previous larger	Monotonic Stack

largest rectangle, histogram, trapping water	Monotonic Stack
sliding window max/min, maximum in every window	Monotonic Queue (Deque)

Linked List Keywords

Keyword / Phrase in Problem	→ Pattern to Use
cycle, loop, circular linked list	Fast and Slow Pointers
middle of linked list	Fast and Slow Pointers
palindrome linked list	Fast and Slow Pointers + Reverse
reverse a linked list	Iterative with prev/curr/next
merge two sorted lists	Two Pointers on linked lists
remove Nth node from end	Two Pointers (gap of N)
LRU Cache	Doubly Linked List + Hash Map

Tree and Graph Keywords

Keyword / Phrase in Problem	→ Pattern to Use
all paths, explore, subtree, depth, height	DFS
lowest common ancestor, path between nodes	DFS
level order, shortest path (unweighted), nearest	BFS
connected components, number of islands, regions	DFS or Union Find
shortest path, minimum cost (weighted graph)	Dijkstra
prerequisites, dependencies, build order, schedule	Topological Sort
cycle in directed graph	Topological Sort or DFS
union, same group, connected, redundant edge	Union Find
autocomplete, prefix search, word search grid	Trie

Optimisation and Enumeration Keywords

Keyword / Phrase in Problem	→ Pattern to Use
minimum/maximum value, optimal, count ways	Dynamic Programming

can we achieve X, is it possible	DP or Greedy
overlapping subproblems (same inputs computed twice)	DP with Memoisation
all combinations, all subsets, generate all	Backtracking
all permutations, all arrangements	Backtracking
N-Queens, Sudoku, word search on grid	Backtracking
intervals, meetings, overlap, merge, schedule	Greedy + Sort by time
minimum rooms, maximum non-overlapping	Greedy + Heap
jump game, minimum steps, greedy choice	Greedy

4.2 Input Structure → Pattern Map

Input Type	First Thought	Common Patterns
1D sorted array	Binary search? Two pointers?	Binary Search, Two Pointers, Sliding Window
1D unsorted array	Sort first? Hash map? DP?	Hash Map, Prefix Sum, Sliding Window, DP
String	Character frequency? Substring?	Sliding Window, Hash Map, Two Pointers, Trie
2D matrix / grid	Regions? Shortest path?	DFS, BFS, Union Find, DP on grid
Linked list	Cycle? Middle? Reverse?	Fast/Slow Pointers, Two Pointers
Binary tree	Subtree result? Level-by-level?	DFS (recursion), BFS (level order)
Graph (edge list)	Shortest path? Components?	BFS, DFS, Dijkstra, Union Find, Topo Sort
Intervals [s,e]	Sort by start. Overlap?	Greedy, Sort, Heap
Numbers (count ways)	Overlapping subproblems?	DP (top-down or bottom-up)
Strings (generate)	All combinations/permutations?	Backtracking

4.3 Constraint-Based Pattern Detection

The size of n reveals what time complexity is required — and that directly tells you which patterns are viable.

Input Size (n)	Maximum Viable Complexity	Patterns That Fit
n ≤ 10	O(n!) or O(2^n)	Brute force, Backtracking all permutations
n ≤ 20	O(2^n)	Backtracking, Bitmask DP
n ≤ 100	$O(n^3)$	3-nested loop DP, Floyd-Warshall
n ≤ 1,000	$O(n^2)$	Nested loops, $O(n^2)$ DP
n ≤ 10,000	$O(n^2)$ borderline	Careful $O(n^2)$ — prefer O(n log n)
n ≤ 100,000	O(n log n)	Sorting, Binary Search, Heap, Divide & Conquer
n ≤ 1,000,000	O(n)	Two Pointers, Sliding Window, Hash Map, Prefix Sum
n > 10,000,000	O(n) or O(log n)	Binary Search, Counting Sort, one-pass algorithms

Key Rule	Look at constraints first. If n ≤ 100,000 you need O(n log n) or better. If n ≤ 1,000,000 you need O(n). This immediately rules out $O(n^2)$ DP or O(2^n) backtracking.

4.4 Pattern Decision Flowchart

Work through these questions in order when you read a new problem. Stop at the first question that gives you a clear answer.

Step 1: Is the input a linked list?
→ **Fast/Slow Pointers (cycle/middle) OR Two Pointers (merge/remove)**

Step 2: Is it a tree or binary tree?
→ **DFS for subtree computation OR BFS for level-order / shortest path**

Step 3: Is it a graph (edges, connections)?
→ **BFS (shortest unweighted) / Dijkstra (weighted) / Topo Sort (dependencies) / Union Find (groups)**

Step 4: Is it a 2D grid / matrix?
→ **DFS for regions/islands OR BFS for shortest path on grid**

Step 5: Is the input sorted (or can be sorted)?
→ **Two Pointers OR Binary Search OR Greedy (intervals)**

Step 6: Does it involve contiguous elements?

→ **Sliding Window OR Prefix Sum**

Step 7: Does it need next greater/smaller?
→ **Monotonic Stack**

Step 8: Does it count frequencies or need O(1) lookup?
→ **Hash Map / Counter / Set**

Step 9: Does it ask for ALL combinations/subsets/perms?
→ **Backtracking**

Step 10: Does it ask for MINIMUM / MAXIMUM / COUNT WAYS?
→ **Dynamic Programming**

Step 11: Is there a greedy choice property?
→ **Greedy**

Step 12: Does it group elements or detect components?
→ **Union Find**

Step 13: Does it involve prefix matching or word search?
→ **Trie**

Pro Tip	Most interview problems are answered by steps 1–6. The deeper steps handle more specialised problems.

4.5 Pre-Coding Mental Checklist

Run through this checklist before writing a single line of code.

Step 1 — Understand the Problem (2 min)

- Read the problem twice. Do not skim.
- Identify: What is the INPUT? What is the OUTPUT?
- What are the CONSTRAINTS? (size of n, value ranges, sorted or not)
- What should I return if the input is empty or has one element?
- Can elements be negative? Can there be duplicates? Is the array sorted?

Step 2 — Work Through Examples (2 min)

- Trace through the provided examples manually.
- Create your own small example (3–5 elements) and solve it by hand.
- Think about edge cases: empty, single element, all same, all negative.
- Ask yourself: Can I see a pattern in how I solved the example by hand?

Step 3 — Identify the Pattern (1 min)

- What is the input structure? (array, string, tree, graph, grid, linked list)
- What does the problem ask? (min/max, count, find all, shortest, yes/no)
- What are the constraints? (n size → required complexity)
- Scan for keywords → match to pattern using Sections 4.1 and 4.2
- No keyword match? Start brute force, then optimise.

Step 4 — Plan Before Coding (1 min)

- State the approach: 'I will use sliding window because...'
- Identify the state: What variables do I need to track?
- Identify the transition: How does the state change at each step?
- Estimate complexity: Time and Space — does it fit the constraints?
- Only start coding once you have a clear plan.

Step 5 — Code and Test (10–15 min)

- Write clean code with simple variable names (left, right, slow, fast, seen).
- Handle the base case / edge case first.
- Test with your small example by tracing through the code manually.
- Check for off-by-one errors (< vs <=, i vs i+1).
- Check for empty input, single element, and edge cases from Step 2.

4.6 Brute Force → Optimise Framework

When you cannot immediately identify the pattern, start with brute force. Then apply these strategies one by one.

Brute Force Issue	Optimisation Strategy	Result
Repeated scanning of array for lookup	Replace list scan with Hash Map	O(n) → O(1) per lookup
Recomputing sum of every subarray	Use Prefix Sum array	$O(n^2) \rightarrow O(n)$

Checking all pairs in sorted array	Use Two Pointers	$O(n^2) \rightarrow O(n)$
Checking all subarrays of size k	Use Sliding Window	$O(nk) \rightarrow O(n)$
Linear scan for sorted data	Use Binary Search	$O(n) \rightarrow O(\log n)$
Recursive tree with repeated calls	Add Memoisation (@lru_cache)	$O(2^n) \rightarrow O(n)$
Checking all subsets	Use DP bottom-up table	$O(2^n) \rightarrow O(n \cdot target)$
Finding max in every window	Use Monotonic Deque	$O(nk) \rightarrow O(n)$
Union/find on naive structures	Union Find with path compression	$O(n) \rightarrow O(\alpha(n)) \approx O(1)$

End of Section 4. Continue with Section 5 — Universal Code Templates.